

Actividad de recuperación día 13



PRODUCTOS, CLIENTES Y PEDIDOS DE VERDAD: `CRUD`, BÓRRADO LÓGICO, TRANSACCIONES Y LOS TRES MENSAJES DEL SERVIDOR

Fecha	Lunes 5 de octubre de 2026
Aprendiz	Sneider Alvarez Santos
Ficha	3428392
Documento	1016717046
Programa	Desarrollo web — SENA
Proyecto	La Carambola Dorada — sistema de gestión de salones de billar
Entorno	XAMPP — Apache 2.4.58 en el puerto 80, PHP 8.1.25, MariaDB 10.4.32
Publicación	Servidor local XAMPP — <code>http://localhost/plan-mejoramiento-desarrollo-web/</code>

Bitácora del día: qué hice

- 1 El **CRUD completo de productos**. Antes el listado venía de la base pero el formulario de alta no guardaba nada, no había edición, y el “eliminar” era un botón de mentira. Ahora las siete operaciones funcionan contra MySQL: crear, leer, actualizar, desactivar, buscar, paginar y ordenar, con el orden validado contra una lista blanca.
- 2 El **CRUD completo de clientes**, en una pantalla nueva (`clientes.php`) con su modelo (`app/modelos/CienteModelo.php`). Aquí sí se borra de verdad, con las dos unicidades del enunciado: documento y correo.
- 3 La **pantalla de pedidos**, otra nueva, con las cuatro líneas de detalle fijas, el listado, el detalle de un pedido y el cambio de estado. El modelo (`app/modelos/PedidoModelo.php`) guarda el pedido y descuenta el inventario dentro de una transacción con bloqueo de filas, y no tiene ninguna función para borrar.
- 4 El **borrado lógico** de productos: tres columnas nuevas en `sql/dia13.sql`, el listado escondido con `WHERE activo = 1`, un filtro por estado y un botón de reactivar.
- 5 Los **tres mensajes del servidor** del enunciado, documentados y demostrados con y sin JavaScript, y las 23 capturas de los tres listados.

Además salieron tres cosas que el enunciado no pedía: la sección 16 de `css/estilos.css` con los estilos nuevos, las dos entradas de menú (Clientes y Pedidos, que dejan el menú en 8, 6 y 3 entradas según el rol) y la clase `.mono`, que se usaba en las tres pantallas y no existía en la hoja de estilos.

Archivos que creé y archivos que toqué

ARCHIVO	ESTADO	PARA QUÉ SIRVE
<code>clientes.php</code>	Nuevo	El punto 2 entero: alta, listado con búsqueda, orden y paginación, edición por <code>?editar=</code> y borrado de verdad. 721 líneas.
<code>pedidos.php</code>	Nuevo	El punto 3 entero: cuatro líneas de detalle, listado, detalle con <code>?ver=</code> y cambio de estado. 879 líneas, y ni una línea de JavaScript.
<code>app/modelos/ClienteModelo.php</code>	Nuevo	Catorce funciones de clientes: búsqueda, orden, paginación, las dos unicidades y el borrado. 421 líneas.
<code>app/modelos/PedidoModelo.php</code>	Nuevo	Nueve funciones: la transacción del alta, el cambio de estado, el detalle y el listado. 513 líneas.
<code>sql/dia13.sql</code>	Nuevo	Los tres cambios de esquema del día 13, como <code>ALTER</code> y no como estructura nueva.
<code>productos.php</code>	Reescrito	El punto 1 y el punto 4. 889 líneas: el alta que ahora guarda, la edición, el filtro por estado y los mensajes del servidor.
<code>app/modelos/ProductoModelo.php</code>	Ampliado	Del día 9 (lectura y búsqueda) a 16 funciones: búsqueda, orden, paginación, estado, auditoría y las validaciones. 520 líneas.
<code>css/estilos.css</code>	Ampliado	La sección 16, con seis apartados: campo, paginación, filtro de estado, cabeceras que ordenan, cabecera del pedido y líneas del pedido.
<code>app/config/menu.php</code>	Ampliado	Dos entradas nuevas (Clientes y Pedidos), con los mismos dos roles que Productos.
<code>app/productos.js</code>	Sin tocar	La revisión del navegador se queda como estaba: es una mejora, y el servidor sigue siendo el que decide.
<code>js/ejercicios.js</code>	Sin tocar	Solo se le quitó una marca BOM al principio, que hacía que el navegador lo tomara por un archivo que no es UTF-8.

Punto 1. Productos: el CRUD que de verdad guarda

El `productos.php` del día 9 pintaba la tabla desde MySQL y se quedaba ahí. El formulario de alta tenía los cuatro campos y un botón que no guardaba nada, y los botones de editar y eliminar no existían. Este día hace las siete operaciones, y cada una dice dónde vive:

OPERACIÓN	CÓMO SE PIDE	DÓNDE SE RESUELVE
Crear	El formulario, con <code>accion=crear</code>	Valida en el servidor y luego <code>insertar()</code> en el modelo. Vuelve al listado con un "Listo".
Leer	El listado, de diez en diez	<code>listarProductos()</code> con el desplazamiento en el <code>LIMIT</code> , no en PHP.

OPERACIÓN	CÓMO SE PIDE	DÓNDE SE RESUELVE
Actualizar	?editar=<id>	El mismo formulario, con accion=editar y el id en un campo oculto.
Eliminar	Un formulario por fila	No borra: desactiva. Es el punto 4, y está en su apartado.
Buscar	?buscar=<texto>	Un LIKE con parámetros preparados sobre código, nombre y categoría.
Ordenar	?orden=<columna>&dir=<asc desc>	La columna se compara contra una lista blanca; si no está, la consulta no la usa.
Paginar	?pagina=<n>	contarProductos() para el total y el LIMIT con desplazamiento para la página.

Por qué el orden no va tal cual a la consulta

Un ORDER BY no se puede poner en una cadena como el resto de los datos, porque el ORDER BY es la única parte de la consulta que MySQL no se puede preparar con un parámetro: no es un valor, es el nombre de una columna. Si el nombre viene del formulario tal cual, quien lo escriba puede poner id; DROP TABLE productos después. Por eso las columnas permitidas están escritas en el modelo:

```
/* app/modelos/ProductoModelo.php */
const COLUMNAS_ORDEN = [
    'codigo'      => 'p.codigo',
    'nombre'      => 'p.nombre',
    'categoria'   => 'c.nombre',
    'valor_alquiler'=> 'p.valor_alquiler',
    'existencias' => 'p.existencias',
    'estado'      => 'p.activo',
];

$columna = COLUMNAS_ORDEN[$pedido] ?? COLUMNAS_ORDEN['nombre'];
$sentencia .= ' ORDER BY ' . $columna . ' ' . $direccion;
```

La dirección solo puede ser ASC o DESC, y se comprueba con un in_array de comparador estricto antes de escribirla en la cadena.

El buscador se hace en la base, no en el navegador

El día 8 tenía un filtro con JavaScript que iba escondiendo filas mientras se escribía. Con la tabla pintada por PHP ese filtro ya no podía funcionar, y no se ha vuelto a poner: lo que se escribe se manda por GET y lo filtra MySQL. Perder el filtrado mientras se teclea no es una pérdida: con la base al otro lado no hay filtrado instantáneo, y el del día 8 solo ocultaba filas de la página que ya estaba cargada, no buscaba entre los 21 productos.

Productos

Inventario de la casa alta, edición, búsqueda, orden y tipo lógico.

Búsqueda: Buscar

Ordenar por: Ordenar

Mostrar: 10 de 10 productos en pantalla

Acciones:

ID	Nombre	Categoría	Stock	Valor de alquiler	Estado	Acciones
ADT-014	Agente de limpieza para interiores	Nombre	12	\$ 1.100	Activo	Editar Eliminar
ESP-012	Servicio de limpieza para exteriores	Nombre	9	\$ 0.800	Activo	Editar Eliminar
CON-018	Conector de conexión de agua	Nombre	2	\$ 0.500	Activo	Editar Eliminar
EST-005	Estación de trabajo de agua	Nombre	0	\$ 0.300	Activo	Editar Eliminar
CON-002	Conector de agua	Nombre	6	\$ 1.100	Activo	Editar Eliminar
QUA-007	Quinto de agua potable	Nombre	7	\$ 0.200	Activo	Editar Eliminar
QUA-003	Quinto de agua	Nombre	6	\$ 1.000	Activo	Editar Eliminar
REC-004	Receptor de agua de lluvia	Nombre	3	\$ 0.500	Activo	Editar Eliminar
ESP-020	Estación de agua potable	Nombre	11	\$ 1.000	Activo	Editar Eliminar
REC-021	Receptor de agua de lluvia	Nombre	1	\$ 0.500	Activo	Editar Eliminar

1 2 3 Siguiente

Nuevo producto

Código: Nombre:

Categoría: Condición:

Existencias: Valor de alquiler (en pesos):

Fecha de ingreso: Proveedor (opcional):

Descripción (opcional):

dia13-productos-listado.png — El listado: buscador, filtro de estado, diez filas por página y las cabeceras que ordenan.

Productos

Inventario de la casa alta, edición, búsqueda, orden y tipo lógico.

Búsqueda: Buscar

Ordenar por: Ordenar

Mostrar: 10 de 10 productos en pantalla

Acciones:

ID	Nombre	Categoría	Stock	Valor de alquiler	Estado	Acciones
ADT-014	Agente de limpieza para interiores	Nombre	12	\$ 1.100	Activo	Editar Eliminar
ESP-012	Servicio de limpieza para exteriores	Nombre	9	\$ 0.800	Activo	Editar Eliminar
CON-018	Conector de conexión de agua	Nombre	2	\$ 0.500	Activo	Editar Eliminar
EST-005	Estación de trabajo de agua	Nombre	0	\$ 0.300	Activo	Editar Eliminar
CON-002	Conector de agua	Nombre	6	\$ 1.100	Activo	Editar Eliminar
QUA-007	Quinto de agua potable	Nombre	7	\$ 0.200	Activo	Editar Eliminar
QUA-003	Quinto de agua	Nombre	6	\$ 1.000	Activo	Editar Eliminar
REC-004	Receptor de agua de lluvia	Nombre	3	\$ 0.500	Activo	Editar Eliminar
ESP-020	Estación de agua potable	Nombre	11	\$ 1.000	Activo	Editar Eliminar
REC-021	Receptor de agua de lluvia	Nombre	1	\$ 0.500	Activo	Editar Eliminar

1 2 3 Siguiente

Editar producto

Está editando: Teco de post 1.45 m. Los cambios se guardan con el mismo botón.

Código: Nombre:

Categoría: Condición:

Existencias: Valor de alquiler (en pesos):

Fecha de ingreso: Proveedor (opcional):

Descripción (opcional):

dia13-productos-editar.png — La edición por ? editar=: el mismo formulario con los datos de la fila.

Nuevo producto

Código: Nombre:

Categoría: Condición:

Existencias: Valor de alquiler (en pesos):

Fecha de ingreso: Proveedor (opcional):

Descripción (opcional):

dia13-productos-formulario.png — El alta. Antes este formulario no guardaba nada.

Productos

Inventario de la casa alta, edición, búsqueda, orden y tipo lógico.

Búsqueda: Buscar

Ordenar por: Ordenar

Mostrar: 10 de 10 productos en pantalla

Acciones:

ID	Nombre	Categoría	Stock	Valor de alquiler	Estado	Acciones
ADT-014	Agente de limpieza para interiores	Nombre	12	\$ 1.100	Activo	Editar Eliminar
ESP-012	Servicio de limpieza para exteriores	Nombre	9	\$ 0.800	Activo	Editar Eliminar
CON-018	Conector de conexión de agua	Nombre	2	\$ 0.500	Activo	Editar Eliminar
EST-005	Estación de trabajo de agua	Nombre	0	\$ 0.300	Activo	Editar Eliminar
CON-002	Conector de agua	Nombre	6	\$ 1.100	Activo	Editar Eliminar
QUA-007	Quinto de agua potable	Nombre	7	\$ 0.200	Activo	Editar Eliminar
QUA-003	Quinto de agua	Nombre	6	\$ 1.000	Activo	Editar Eliminar
REC-004	Receptor de agua de lluvia	Nombre	3	\$ 0.500	Activo	Editar Eliminar
ESP-020	Estación de agua potable	Nombre	11	\$ 1.000	Activo	Editar Eliminar
REC-021	Receptor de agua de lluvia	Nombre	1	\$ 0.500	Activo	Editar Eliminar

1 2 3 Siguiente

Nuevo producto

Código: Nombre:

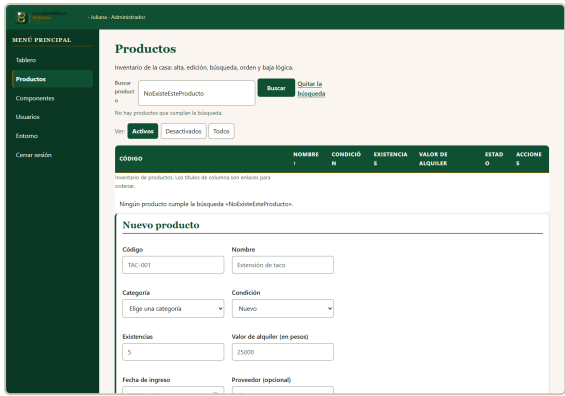
Categoría: Condición:

Existencias: Valor de alquiler (en pesos):

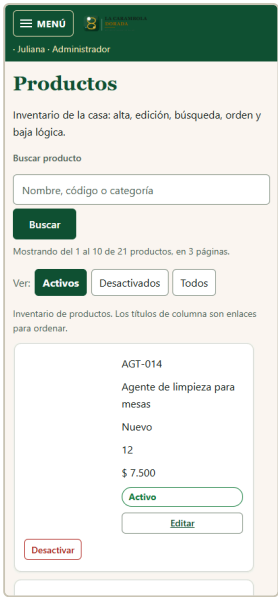
Fecha de ingreso: Proveedor (opcional):

Descripción (opcional):

dia13-productos-busqueda.png — Búsqueda por texto: el filtro lo puso MySQL, no el navegador.



dia13-productos-sin-resultados.png — Sin resultados: avisa y no inventa filas.



dia13-productos-movil.png — A 390 px la tabla se desliza con `.tabla-scroll` y el buscador pasa a dos columnas.

Punto 2. Clientes: el mismo CRUD con dos unicidades

`clientes.php` es la misma estructura del punto 1 aplicada a otra tabla, con las dos reglas que el enunciado pide solo para clientes:

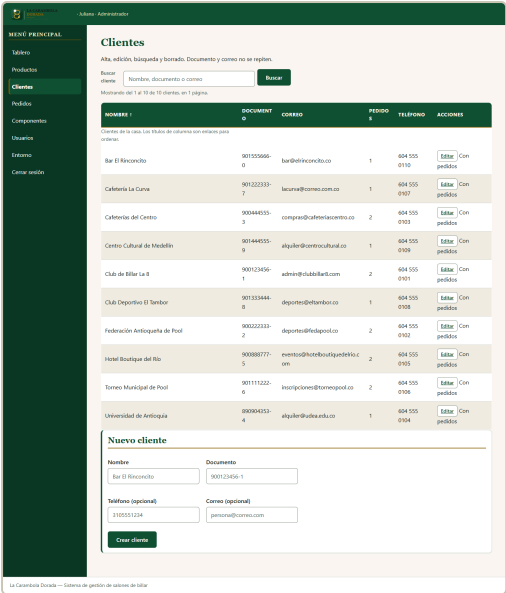
REGLA	LO QUE SE COMPRUEBA	POR QUÉ NO BASTA CON EL ÍNDICE
Documento único	Antes de guardar, una consulta busca el mismo documento en otro cliente	El índice <code>UNIQUE</code> de MySQL lo garantiza, pero su error (1062) dice "duplicate entry", que no le sirve de nada a quien está delante de la pantalla.
Correo único	Lo mismo con el correo	Dos clientes con el mismo correo harían que una sola respuesta de confirmación llegase a los dos.

Las dos reglas existen además como índice `UNIQUE` en la base. Esa es la combinación que importa: la comprobación del PHP es para que el mensaje sea legible, y el índice es para que la base no deje pasar el duplicado aunque el código tenga un fallo. Si las dos se pueden contradecir, el índice gana, porque es el que no se equivoca.

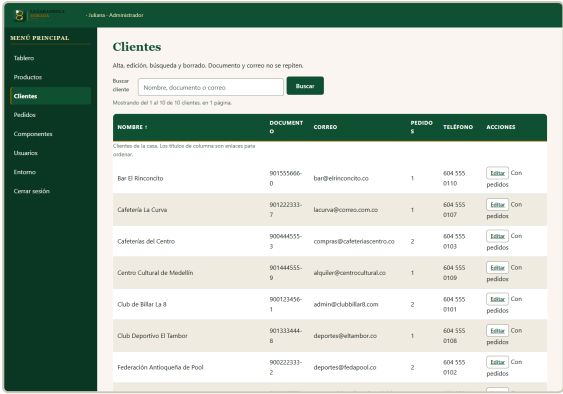
Por qué aquí sí se borra y en productos no

El punto 4 pide borrado lógico para los **productos**, y el listado de clientes lo respeta: un cliente no se desactiva, ni tiene columnas `activo` ni `desactivado_en`. Un cliente es distinto de un producto: un producto es una oferta que puede dejar de existir, y un cliente es una persona.

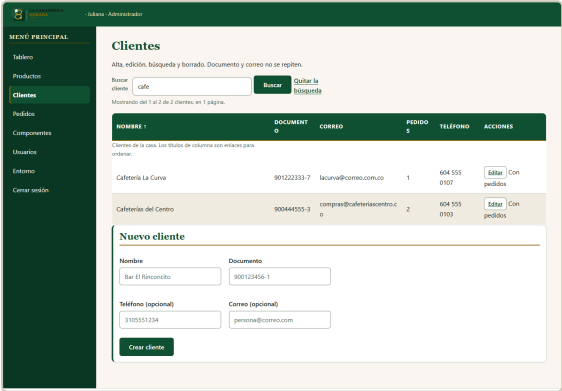
La página borra de verdad, y la base dice que no cuando no se puede. `clientes` está en `pedidos` con `ON DELETE RESTRICT`, de modo que un cliente con pedidos no se puede borrar ni queriendo. Antes de enviar el `DELETE` la página pregunta cuántos pedidos tiene y, si no son cero, no lo envía: se lo dice a la persona y no se pierde nada. Esa pregunta no sustituye al índice, es la diferencia entre un error y un mensaje.



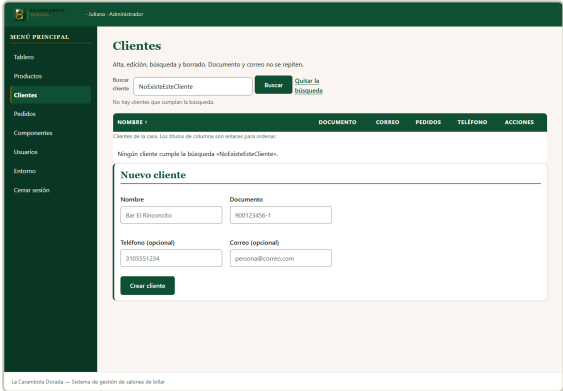
dia13-clientes-listado.png — El listado de clientes, con la misma búsqueda, orden y paginación.



dia13-clientes-editar.png — La edición. El documento no se deja cambiar si el cliente ya tiene pedidos.



dia13-clientes-busqueda.png — La búsqueda también mira el documento y el correo, no solo el nombre.



dia13-clientes-sin-resultados.png — Sin resultados.

dia13-clientes-movil.png — El listado de clientes a 390 px.

Punto 3. Pedidos: cuatro líneas, transacción y ningún borrar

Esta pantalla no se parece en nada a las otras dos, y por eso conviene decir de entrada lo que tiene de distinto:

- **Cuatro líneas fijas en el HTML.** El enunciado pide cuatro líneas de detalle y están las cuatro escritas, siempre. No hay líneas escondidas esperando a que un script las rellene, ni un botón de “añadir línea” que sin JavaScript no hace nada: con el JavaScript apagado el pedido se hace igual.
- **El precio no viene del formulario.** El `<input type="hidden">` del precio no existe. Al guardar, el modelo lee `productos.valor_alquiler` de la base. Si el precio fuera un campo del formulario, quien envie la página a mano pondría el precio que quisiera.
- **No hay borrar.** No es un olvido: el modelo no tiene ninguna función que borre un pedido. Un pedido es un hecho que pasó; las mesas salieron y el total está en las facturas.

El alta, línea a línea

El total se calcula con los precios leídos de la base, no con lo que venía el navegador: por eso el total que se guarda y el que se pinta son el mismo número, y por eso dos personas que miren la misma línea ven lo mismo.

dia13-pedidos-listado.png — El listado de pedidos, con el estado como enlace de filtro.

dia13-pedidos-detalle.png — El detalle con las líneas, el cambio de estado y el total.

dia13-pedidos-busqueda.png — La búsqueda mira el número, el cliente, el documento y el estado.

dia13-pedidos-orden.png — Ordenado por total, con la flecha en la cabecera.

dia13-pedidos-paginacion.png — La paginación con la página actual marcada con `aria-current`.

dia13-pedidos-movil.png — A 390 px.

dia13-pedidos-formulario.png — Las cuatro líneas de detalle, fijas en el HTML. No hay ninguna línea que aparezca con JavaScript.

Por qué una transacción y no dos consultas

Un pedido descuenta existencias. Si el `INSERT` del pedido se guardara y la actualización del inventario fallara, quedaría un pedido con cuatro productos que la base no tiene. Y al revés: si se descontara primero y el pedido no se guardara, el inventario estaría corto sin que haya constancia de por qué. Las dos cosas van dentro de `beginTransaction()`, y si algo lanza una excepción se llama a `rollback()`.

```
$pdo->beginTransaction();

/* FOR UPDATE: el segundo que llegue a este SELECT se queda esperando a que el
   primero termine. Sin esto, dos pedidos a la vez pueden leer las mismas 3
   existencias, los dos creen que hay, y el inventario queda en -1. */
$fila = $pdo->query("SELECT existencias FROM productos WHERE id = ... FOR UPDATE")-
>fetch();

if ($fila['existencias'] < $unidades) {
    throw new RuntimeException('No hay stock suficiente.');
```

La comprobación de existencias se hace **dentro** de la transacción, con el bloqueo puesto. Fuera de ella la respuesta sería en función de un dato que ya puede haber cambiado.

Los tres estados

El `ENUM` de `pedidos.estado` tiene tres valores: `Pendiente`, `Confirmado` y `Entregado`. El cambio de estado es un formulario por pedido, con su propio token, y solo admite el estado

que le toca: un pedido `Entregado` no vuelve a `Pendiente`, porque un pedido entregado no se desentrega.

Punto 4. El borrado lógico de productos

Por qué no se borra de verdad

`productos` está en `detalle_pedidos` con `ON DELETE RESTRICT`. Los quince pedidos que hay en la base son de días anteriores y se apoyan en esa integridad: si un producto desaparece de verdad, los pedidos antiguos se quedan sin el nombre que se llevaron. Y un producto que no estuvo en ningún pedido, ese sí se podría borrar, pero entonces no volvería nunca. En los dos casos lo que corresponde es la baja lógica.

Lo que se añadió a la tabla

```
-- sql/dia13.sql
ALTER TABLE productos
  ADD COLUMN activo          TINYINT(1)  NOT NULL DEFAULT 1,
  ADD COLUMN desactivado_en  DATETIME     NULL,
  ADD COLUMN desactivado_por INT          NULL,
  ADD FOREIGN KEY (desactivado_por) REFERENCES usuarios(id) ON DELETE SET NULL;
```

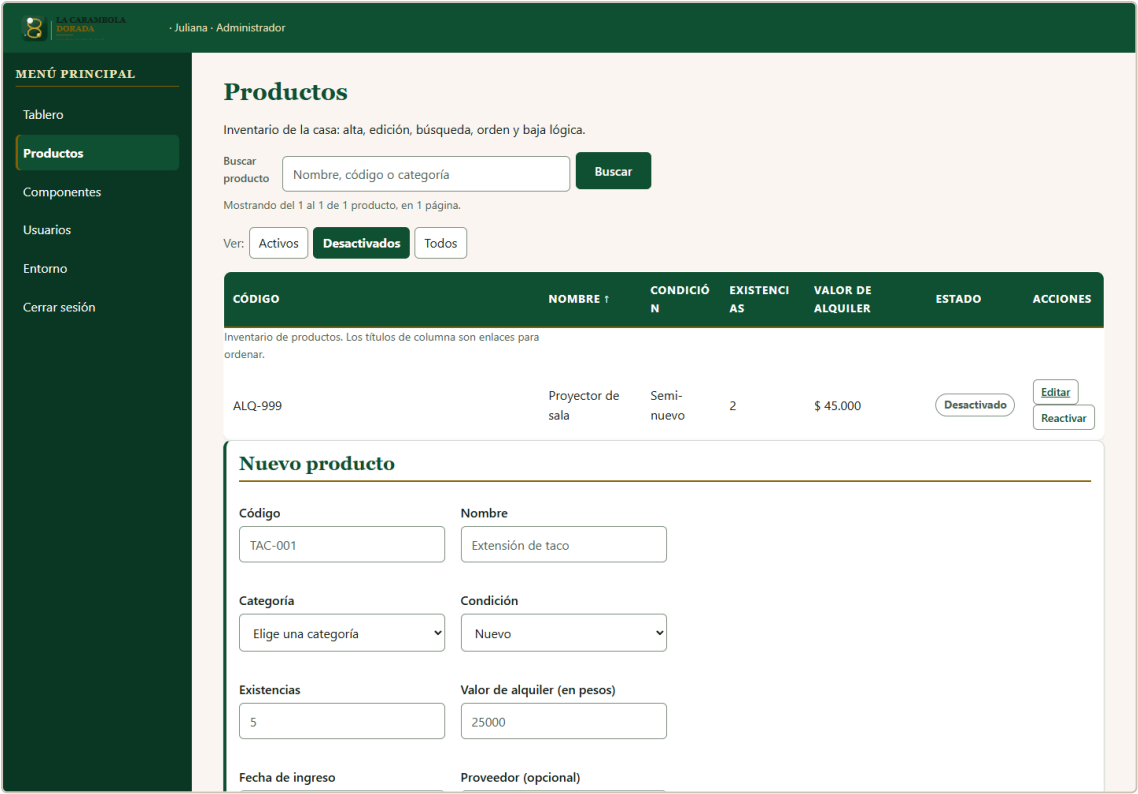
Las tres columnas van juntas y por eso van en el mismo `ALTER`. Un producto desactivado sin fecha es un producto que “nunca existió”, y no hay forma de auditarlo ni de saber quién lo apagó; por eso la fecha y el usuario no son un extra:

- `activo = 1` es lo que ve el listado por defecto: el `WHERE` del filtro, no un `IF` en la plantilla.
- `desactivado_en` es *cuándo*.
- `desactivado_por` es *quién*, y se puede volver a poner a `NULL` al reactivar.

El filtro son enlaces, no un menú desplegable

El estado se filtra con tres enlaces (`?estado=activos`, `inactivos`, `todos`) y no con un `<select>` que se envía con JavaScript. Los enlaces se pulsan con el teclado, funcionan sin JavaScript y además la URL se puede compartir: si alguien está mirando los productos desactivados, la puede mandar tal cual. El que está activo se marca con `aria-current="page"`.

Un detalle que se ve en el menú y no en el listado: el buscador del formulario de pedido y el de la edición sí ofrecen productos activos, pero el detalle de un pedido antiguo enseña el producto aunque hoy esté desactivado, con la etiqueta de “desactivado”. Un producto que se apagó el Martes sigue siendo el que se llevó el Lunes, y un pedido no se reescribe.



dia13-productos-desactivados.png — El filtro de estado. Los productos desactivados siguen en la base, con su fecha y su usuario, y el botón dice “Reactivar” en vez de “Desactivar”.

Punto 5. Los tres mensajes del servidor

El enunciado pide tres casos de validación, y lo que importa es *quién* los escribe. El formulario lleva `novalidate` a propósito: con él, el navegador no para el envío ni pinta su burbuja, el `POST` llega entero al servidor y es el servidor el que contesta. La revisión de `app/productos.js` se queda, pero añade: nunca quita la del servidor, y si el JavaScript está apagado la imagen es idéntica.

CASO	LO QUE SE ENVÍA	LO QUE CONTESTA EL SERVIDOR
1. Campos vacíos	El formulario entero sin escribir nada	“El código del producto es obligatorio.” · “El nombre del producto es obligatorio.” · “Hay que elegir una categoría.” · “Las existencias son obligatorias.” · “El valor de alquiler es obligatorio.” · “La fecha de ingreso es obligatoria.”
2. Precio negativo	<code>valor_alquiler = -5000</code>	“El valor de alquiler tiene que ser mayor que cero: <code>-5000</code> no es un precio.” El número que se escribió vuelve en el mensaje.
3. Texto en un campo numérico	<code>existencias = hola</code>	“Las existencias tienen que ser un número entero. Escribiste: «hola».” Lo escrito vuelve en el mensaje, también con comillas.

Los tres se pueden ver **enteros** en `assets/img/evidencia/dia13-mensajes-servidor.txt`, que no está escrito a mano: es la salida de una corrida que envía los tres formularios por HTTP y copia lo que el servidor respondió.

Cuando un formulario vuelve con errores, el **old input** conserva lo que se había escrito. Volver a escribir veinte letras porque se saltó un campo es la forma más rápida de perder a la gente en un formulario. El token CSRF se pide de nuevo, así que el guardado dos veces no es un problema.

La Carambola Dorada — Sistema de gestión de salones de billar

`dia13-productos-js-error.png` — Los mensajes con JavaScript encendido. La lista de arriba es la que pinta el servidor.

La misma prueba sin JavaScript

Con el JavaScript del navegador desactivado desde el propio navegador, los tres mensajes salen igual: eso demuestra que la validación no estaba en el script. En pedidos, además, se ven las cuatro líneas de detalle y el error de cada línea junto a su campo, porque las líneas están en el HTML y el error lo pinta el servidor.

LA CARAMBOLA DORADA · Juliana · Administrador

MENÚ PRINCIPAL

- Tablero
- Productos**
- Componentes
- Usuarios
- Entorno
- Cerrar sesión

Productos

Inventario de la casa: alta, edición, búsqueda, orden y baja lógica.

No se pudo hacer: No se guardó el producto. Revisa lo que está marcado en rojo.

Buscar producto

Mostrando del 1 al 10 de 21 productos, en 3 páginas.

Ver:

CÓDIGO	NOMBRE ↑	CONDICIÓN	EXISTENCIAS	VALOR DE ALQUILER	ESTADO	ACCIONES
AGT-014	Agente de limpieza para mesas	Nuevo	12	\$ 7.500	Activo	Editar Desactivar
REP-012	Banda de goma para pocket	Nuevo	9	\$ 6.000	Activo	Editar Desactivar
CON-013	Conector de extensión	En reparación	2	\$ 9.000	Activo	Editar Desactivar
EXT-005	Extensión de taco	En reparación	0	\$ 10.000	Activo	Editar Desactivar
SOP-022	Extensora rígida	Nuevo	6	\$ 11.000	Activo	Editar Desactivar
GLIA-007	Gautes de cuero	Semi-	7	\$ 22.000	Activo	Editar

Inventario de productos. Los títulos de columna son enlaces para ordenar.

dia13-productos-sin-js-error.png — Sin JavaScript: los mismos mensajes, en el mismo sitio.

LA CARAMBOLA DORADA · Juliana · Administrador

Cafetería La Curva (901222333-7)

PRODUCTO	UNIDADES
Líneas del pedido. Las que se dejen vacías se cuentan como cero.	
Línea 1 TAC-001 — Taco de poc	Unidades de la línea 1 much
Línea 2 EXT-005 — Extensión d	Unidades de la línea 2 0
Línea 3 — Ninguno —	Unidades de la línea 3 1
Línea 4 — Ninguno —	Unidades de la línea 4 1

La Carambola Dorada — Sistema de gestión de salones de billar

dia13-pedidos-sin-js-error.png — El alta de pedido sin JavaScript: se eligió el cliente y los productos con el teclado, se escribieron las unidades a mano y el servidor respondió «Las unidades tienen que ser un número entero» y «Las unidades tienen que ser al menos una», cada uno en su línea.

La Carambola Dorada — Sistema de gestión de salones de billar

dia13-clientes-sin-js-error.png — El mismo resultado en clientes, con las dos unicidades comprobadas por el servidor.

Los problemas que me salieron y cómo los resolví

Un mensaje global se comía los errores de cada línea

En `pedidos.php` había un mensaje de resumen que decía “al menos un producto”. Cuando el servidor devolvía errores de línea, ese mensaje salía encima y tapaba los errores buenos: la persona veía “al menos un producto” y no ve que lo que escribió en la línea 2 no es un número. Lo que se buscó no era el texto, sino si la lista de errores venía vacía, y una lista vacía y una lista con claves vacías no se comparan igual:

```
/* antes: [] !== ['unidades-1' => '', 'unidades-2' => 'no es un numero'] -> true */
/* despues: las claves que tienen algo de verdad */
$hayErrores = count(array_filter($errores, static fn($e) => $e !== '')) > 0;
```

Con eso, el aviso global solo aparece cuando de verdad no hay ninguna línea con error, y los errores por línea salen siempre.

Un número malo y una cantidad mala.tenían el mismo mensaje

Escribir `muchas` unidades y escribir `-3` salía con el mismo texto, “las unidades tienen que ser un número entero”. Y con `-3` el mensaje es además falso: `-3` es un número entero. Son dos equivocaciones distintas: una es no saber escribir el número y la otra es no saber que no se puede pedir en negativo. Ahora cada una tiene el suyo (“sin letras ni signos”, “no pueden ser negativas”), y el del número de unidades vuelve con lo que se escribió.

Mi primera prueba HTTP destruía un pedido de la semilla

La primera versión de la prueba de pedidos creaba un pedido, comprobaba el inventario y al final `DELETE` aba “los pedidos de prueba”. La condición era por código, no por identificador, y como el pedido de prueba heredaba un código parecido a uno de la semilla, se llevó por

delante el pedido 15, que es de días anteriores. Se restauró a mano, con el pedido, sus tres líneas y las existencias de los dos productos que había tocado, y la prueba quedó como está:

```
$maxIdAlEmpezar = (int) $pdo->query('SELECT MAX(id) FROM pedidos')->fetchColumn();
/* ... */
$pdo->prepare('DELETE FROM pedidos WHERE id > ?')->execute([$maxIdAlEmpezar]);
```

La regla nueva es que la prueba solo toca lo que ella misma creó, y que además devuelve el inventario a como estaba. Una prueba que ensucia la base no es una prueba: es un cambio.

Un clic que no llegó a ninguna parte

Para capturar el mensaje de “pedido guardado” se pulsó el botón de enviar con el ratón, y no pasó nada: el botón está al final de la página y la coordenada del clic se calculó sobre la ventana, no sobre el documento entero, así que el clic se iba a la pantalla de al lado. La solución es lo que haría una persona: `Enter` dentro del último campo.

La clase `.mono` que se usaba y no existía

Las tres pantallas marcan con `class="mono"` los datos que son un código y no una frase: el número de pedido, el documento, un teléfono, un correo, un precio. Buscando la clase en `css/estilos.css` resultó que no estaba: se usaba en la página y nadie la había definido, así que esos datos salían con la fuente del texto. Se añadió con la misma tipografía que ya usaba `pre.codigo`.

Dos secciones 16.3 en la hoja de estilos

El filtro de estado y las cabeceras que ordenan se numeraron las dos como 16.3, porque la sección se añadió por partes. Renumerada del 16.1 al 16.6, que es lo que se lee en un archivo de estilos para saber si falta algo.

Caracteres que se colaron en los comentarios

Al escribir `pedidos.php` se colaron en los comentarios unos caracteres chinos y cirílicos, de los que entran al escribir rápido y el teclado se queda en otro idioma. No se ven ni rompen nada, pero no son español. Se detectaron con un escáner que avisa de cualquier letra que no esté en el alfabeto español, y se reescribieron. El mismo escáner encontró una marca BOM al principio de `js/ejercicios.js`, que hace que el navegador tome el archivo como si no fuera UTF-8 y fue quitada.

La evidencia salió vacía y elCulpa no era de la página

La primera vez que se recogieron los tres mensajes del servidor por HTTP, la lista de errores salió vacía y no había ninguna alerta. La página estaba bien: el script de la prueba se había olvidado del campo oculto `accion`, que es lo que dice si el `POST` es un alta o una edición. Sin ese campo el servidor no hace nada y vuelve a pintar el formulario, que es justo lo que se vio. Se añadió el campo y los tres casos salieron a la primera.

Los criterios de evaluación, punto por punto

LO QUE PIDE EL ENUNCIADO	ESTADO	DÓNDE SE PUEDE VER
CRUD de productos, con búsqueda, paginación y orden	<code>Sí</code>	<code>productos.php</code> y <code>ProductoModelo.php</code> ; el orden validado contra la lista blanca de columnas

LO QUE PIDE EL ENUNCIADO	ESTADO	DÓNDE SE PUEDE VER
CRUD de clientes, con documento y correo únicos	Si	<code>clientes.php</code> ; las dos comprobaciones antes de guardar y los dos índices <code>UNIQUE</code>
Pedido con cuatro líneas de detalle	Si	<code>pedidos.php</code> : las cuatro líneas están en el HTML, sin JavaScript
El pedido descuenta el inventario	Si	<code>crearPedidoConDetalle()</code> , dentro de la transacción y con <code>FOR UPDATE</code>
El precio sale de la base, no del formulario	Si	El modelo lee <code>valor_alquiler</code> ; en el formulario no hay ningún campo de precio
Un pedido no se borra	Si	El modelo no tiene ninguna función de borrado, y la página no pinta el botón
Borrado lógico de productos	Si	<code>sql/dia13.sql</code> : <code>activo</code> , <code>desactivado_en</code> y <code>desactivado_por</code>
Un producto desactivado sigue en los pedidos antiguos	Si	El detalle lo muestra con la etiqueta; solo el buscador del alta exige <code>activo = 1</code>
Los tres mensajes de validación	Si	<code>dia13-mensajes-servidor.txt</code> y las dos capturas con y sin JavaScript
Las páginas siguen protegidas por rol	Si	<code>exigirRol('administrador', 'vendedor')</code> en las tres, en la primera instrucción
El menú cambia con el rol	Si	8 entradas el administrador, 6 el vendedor, 3 el consultor

Cómo se verificó

Sintaxis de los 32 archivos PHP

C:\xampp\php\php.exe -l (cada archivo .php del proyecto)
→ 32 revisados, 0 con errores de sintaxis

Código de respuesta de cada página sin sesión

PÁGINA	CÓDIGO	QUÉ SIGNIFICA
<code>login.php</code> , <code>registro.php</code>	200	Públicas, como deben ser.
<code>dashboard.php</code> , <code>productos.php</code> , <code>clientes.php</code> , <code>pedidos.php</code> , <code>usuarios.php</code> , <code>componentes.php</code> , <code>phpinfo.php</code>	302	El guardián devuelve al ingreso.
<code>pedidos.php?ver=1</code> , <code>clientes.php?editar=1</code> , <code>productos.php?editar=1</code>	302	Con parámetro también: la protección no es solo del menú.
<code>salir.php</code>	405	Con un GET no cierra nada: solo acepta el POST del formulario.

Las tres pruebas de extremo a extremo

Cada una entra con la sesión de verdad, hace el recorrido completo por HTTP y comprueba lo que tiene que comprobar, y además que al terminar no queda nada en la base:

http-estado-productos.php	→	TODO CORRECTO	(alta, edición, búsqueda, orden, paginación, desactivación, tres mensajes)
http-estado-clientes.php	→	TODO CORRECTO	(las dos unicidades, la edición, el borrado
http-estado-pedidos.php	→	TODO CORRECTO	restringido y el 403 del consultor)
			(alta con stock, rollback sin stock, los tres
			estados, la paginación y el 403 del consultor)

La base, comparada con la semilla

```

productos comparados: 21, diferencias: 0
productos           21
productos activos  21
clientes           10
pedidos            15
líneas             35
usuarios           7
pedidos con el total descuadrado: 0

→ LA BASE ESTA COMO EL SEED

```

La comparación no es un recuento: compara las 21 filas de `productos` campo por campo con `sql/datos.sql`, y después comprueba que cada pedido tenga su total igual a la suma de sus líneas. Un total descuadrado sería la señal de que el precio se guardó desde el navegador en lugar de desde la base, y por eso se mira.

El menú visible por rol

Las dos entradas nuevas están en el mismo bloque de `app/config/menu.php` que Productos, con los mismos dos roles. Las cuentas salen de leer ese archivo, no de contarlas a mano:

```

administrador  8 entradas: Tablero, Productos, Clientes, Pedidos, Componentes, Usuarios,
Entorno, Cerrar sesión
vendedor      6 entradas: Tablero, Productos, Clientes, Pedidos, Componentes, Cerrar
sesión
consultor     3 entradas: Tablero, Componentes, Cerrar sesión

```

El menú esconder Clientes y Pedidos al consultor no le impide entrar por la URL, y por eso las tres páginas vuelven a llamar a `exigirRol()`. El menú es para no tener que leer; el 403 lo contesta la página.

Los tres mensajes, por HTTP

`assets/img/evidencia/dia13-mensajes-servidor.txt` y `assets/img/evidencia/dia13-menu-por-rol.txt` son las dos salidas completas de las comprobaciones, guardadas como texto para que se puedan leer sin abrir la base.

Las 23 capturas

Nueve de productos, seis de clientes y ocho de pedidos, todas en `assets/img/evidencia/` con el prefijo `dia13-`: listado, formulario, edición, búsqueda, sin resultados, desactivados, los tres mensajes con y sin JavaScript, y la vista de 390 px de cada listado. Las que el enunciado pide expressly son las de los tres mensajes (`dia13-productos-js-error.png`) y `dia13-productos-`

`sin-js-error.png`); las demás están porque una bitácora sin capturas de la búsqueda y de la vista móvil no demuestra el CRUD entero.

Sneider Alvarez Santos

Documento 1016717046 — Ficha 3428392

La Carambola Dorada — Actividad de recuperación, día 13

5 puntos · 23 capturas